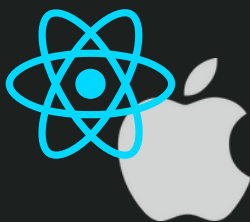


— por
Vinícius
Guedes



Bridge React Native

Conceitos de conexões nativas
entre **React Native e iOS**



SOBRE O AUTOR

Sou o Vinícius Guedes especialista em React Native com mais de 10 anos de experiência no desenvolvimento de aplicativos móveis cross-platform e nativos. Ao longo de minha carreira, tenho atuado na criação e otimização de soluções móveis robustas, combinando conhecimento profundo em tecnologias modernas com uma abordagem prática e orientada para resultados. Minha expertise abrange desde a construção de interfaces de usuário sofisticadas até a integração de funcionalidades nativas complexas, que me renderam conhecimentos em palestras e desafios no campo do desenvolvimento móvel.

Este ebook é fruto de minha experiência e conhecimento na área, e reflete meu compromisso em capacitar desenvolvedores com técnicas e práticas avançadas para criar aplicações móveis de alta qualidade.

O QUE IREMOS APRENDER

Neste ebook, vamos explorar como estabelecer uma conexão eficaz entre o React Native e o código nativo em iOS usando uma bridge. Vamos além do Objective-C, iremos trabalhar também com o Swift. Através de uma abordagem prática e detalhada, você irá aprender:

- **Chamar uma Função Simples:**
 - Objetivo: Aprender a expor e chamar funções básicas do código Swift a partir do React Native.
 - Detalhes: Vamos implementar uma função simples em Swift e chamá-la diretamente do JavaScript, demonstrando a integração básica entre os dois mundos.
- **Enviar Parâmetros para uma Função:**
 - Objetivo: Entender como passar parâmetros do React Native para o código Swift.
 - Detalhes: Iremos criar uma função para aceitar parâmetros e testar o envio de dados do React Native para o código nativo, evidenciando a comunicação bidirecional.
- **Retornar Valores de uma Função:**
 - Objetivo: Aprender a retornar valores do código nativo para o React Native.
 - Detalhes: Criaremos a lógica para que o código Swift retorne um valor para o JavaScript, utilizando Promises para lidar com a resposta assíncrona.
- **Criar um Listener de Eventos:**
 - Objetivo: Implementar e gerenciar eventos emitidos do código nativo para o React Native.
 - Detalhes: Iremos criar um listener no React Native para escutar eventos emitidos pelo código Swift e mostrar como reagir a esses eventos em tempo real.

SUMÁRIO

Introdução	5
Pré requisitos	6
Projeto	7
MóduloNativo	10
Conexão React Native	23
Envio de Parâmetros	27
Retorno de Função	31
Event Listener	36
Resumo	42

INTRODUÇÃO

No universo do desenvolvimento de aplicativos móveis, o React Native se consolidou como uma das ferramentas mais poderosas para a criação de aplicativos cross-platform. Ele permite que os desenvolvedores escrevam um único código em JavaScript que roda tanto no Android quanto no iOS, otimizando o tempo de desenvolvimento e manutenção. No entanto, existem momentos em que você precisa acessar funcionalidades específicas de uma plataforma que não estão disponíveis diretamente no React Native. É aqui que o conceito de Bridge entra em ação.

A Bridge é uma ponte entre o código JavaScript e os módulos nativos, permitindo que você interaja diretamente com as APIs nativas do iOS ou Android. No caso do iOS, ao construir uma bridge, você pode acessar recursos e funcionalidades que estão além do escopo do React Native, utilizando Objective-C e/ou Swift. Isso oferece uma flexibilidade incomparável para criar soluções sob medida, como acesso a bibliotecas específicas do sistema, ou até mesmo otimizações de desempenho.

Compreender como construir uma bridge é uma habilidade essencial para qualquer desenvolvedor React Native que deseje se destacar. Embora o React Native ofereça soluções que cobrem a maior parte das necessidades de um projeto, saber integrar módulos nativos abre um mundo de possibilidades. Um desenvolvedor que domina a criação de bridges pode:

- Melhorar o desempenho de aplicativos que demandam processamento intensivo.
- Integrar funcionalidades que ainda não têm suporte oficial no React Native.
- Construir módulos reutilizáveis que podem ser aproveitados por outros desenvolvedores.

Este conhecimento também expande suas oportunidades de carreira, pois você será capaz de enfrentar desafios mais complexos e técnicos, tornando-se um desenvolvedor mais completo e preparado para projetos maiores e mais sofisticados.

Espero que você tenha um grande proveito deste eBook e que seja um diferencial para sua carreira. Este eBook vai guiá-lo passo a passo através desse processo, facilitando sua jornada para desbloquear todo o potencial do desenvolvimento nativo dentro do React Native. Vamos começar?

PRÉ REQUISITOS

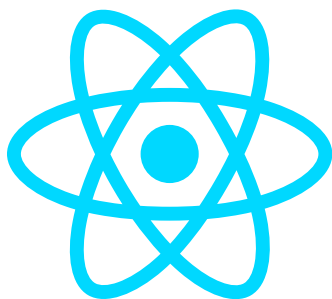
Antes de mergulharmos no processo de criação de bridges no iOS, é importante que você já tenha um entendimento sólido de alguns conceitos e ferramentas:

- **React Native:** Ambiente de desenvolvimento React Native configurado.
- **JavaScript:** Como os módulos nativos se comunicam via JavaScript, é essencial que você esteja confortável com essa linguagem.
- **Sistema Operacional MacOS:** Para desenvolver ou dar manutenção para qualquer aplicativo iOS, é necessário ter um computador que suporte o sistema operacional MacOS, como um Macbook.

O propósito deste curso é te fazer **entender os conceito de conexões bridge**. Para isso, escolhi usar o Expo Bare Workflow por se tratar da maneira "menos comum" de se criar um projeto de alta escala. Assim, quero te fazer entender que independe de React Native CLI ou Expo (desde que seja Bare Workflow) você consegue realizar conexões com módulos nativos.

Projeto React Native

CAPÍTULO 1



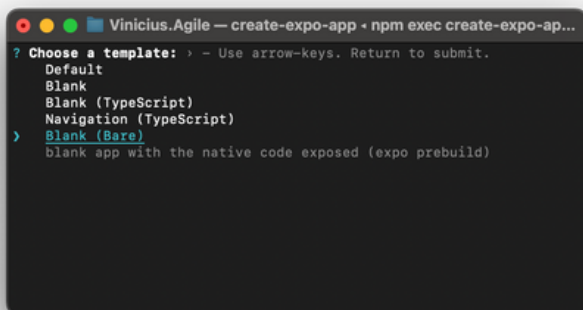
PROJETO REACT NATIVE

Na página anterior, explicamos o motivo de se usar o Expo, então vamos agora criar nosso projeto.

Em seu terminal navegue até a pasta em que deseja criar o seu app, e execute o comando abaixo para criarmos nosso app.

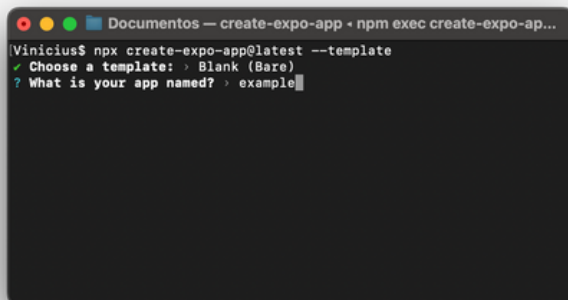
```
npx create-expo-app@latest --template
```

Você terá uma saída como a seguinte:



```
Vinicius.Agile — create-expo-app • npm exec create-expo-ap...
? Choose a template: > - Use arrow-keys. Return to submit.
  Default
  Blank
  Blank (TypeScript)
  Navigation (TypeScript)
> Blank (Bare)
  blank app with the native code exposed (expo prebuild)
```

Escolha a opção "Blank (Bare)". Em seguida, será perguntado sobre qual nome você deseja dar ao seu app, para este exemplo nomeamos nosso app como example.



```
Vinicius$ npx create-expo-app@latest --template
✓ Choose a template: > Blank (Bare)
? What is your app named? > example
```


Após isso, temos nosso projeto disponível para trabalharmos. Mas antes de seguir podemos analisar as versões atuais do nosso projeto. Para isso, abra seu projeto no Visual Studio Code e em seguida abra o **package.json**.

Para este ebook, iremos trabalhar com as seguintes versões:

react: 18.3.1

react-native: 0.75.3



```
package.json > ...
You, 10 seconds ago | 1 author (You)

1  {
2    "name": "example",
3    "version": "0.0.1",
4    "private": true,
5    "scripts": {
6      "android": "react-native run-android",
7      "ios": "react-native run-ios",
8      "lint": "eslint .",
9      "start": "react-native start",
10     "test": "jest"
11   },
12   "dependencies": {
13     "react": "18.3.1",
14     "react-native": "0.75.3"
15   },
16   "devDependencies": {
17     "@babel/core": "^7.20.0",
18     "@babel/preset-env": "^7.20.0",
19     "@babel/runtime": "^7.20.0",
20     "@react-native/babel-preset": "0.75.3",
21     "@react-native/eslint-config": "0.75.3",
22     "@react-native/metro-config": "0.75.3",
23     "@react-native/typescript-config": "0.75.3",
24     "@types/react": "^18.2.6",
25     "@types/react-test-renderer": "^18.0.0",
26     "babel-jest": "^29.6.3",
27     "eslint": "^8.19.0",
28     "jest": "^29.6.3",
29     "prettier": "2.8.0",
30     "react-test-renderer": "18.3.1",
31     "typescript": "5.0.4"
32   },
33   "engines": {
34     "node": ">=18"
35   }
36 }
37
```

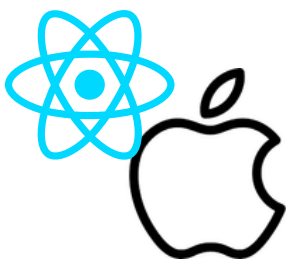
Caso as versões do projeto em que você criou estejam diferentes, não se preocupe. O que você aprenderá neste ebook servirá para versões posteriores e até mesmo para versões anteriores. A intenção de mostrar as versões é apenas para fins didáticos, de modo que você possa comparar com as suas.

Agora que temos nosso app criado. Vamos para o que interessa.

Let's Go!

Módulo Nativo

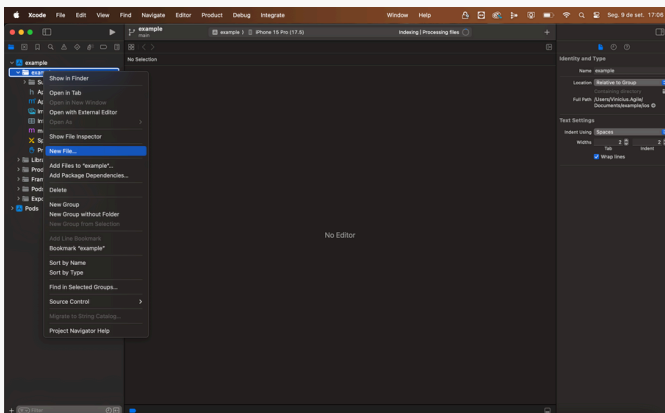
CAPÍTULO 2



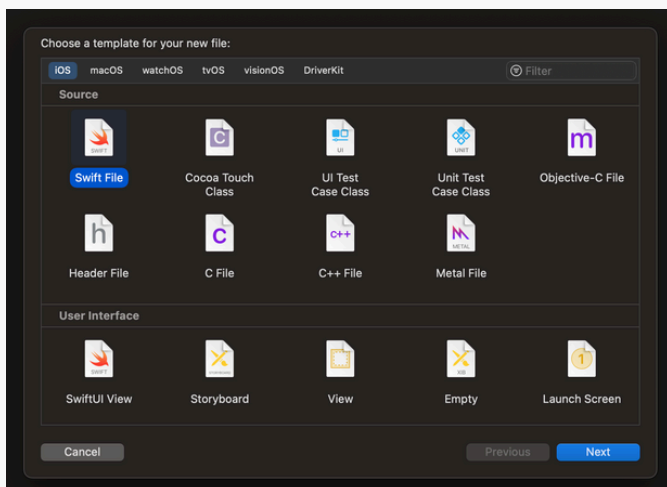
Bridge iOS

2. MÓDULO NATIVO

1. Em SEU_PROJETO/ios Abra o arquivo SEU_PROJETO.xcworkspace.
2. Dentro do Xcode, entre na pasta SEU_PROJETO/SEU_PROJETO e clique com o botão direito do mouse, após isso, selecione a opção "New File...". Confira na imagem abaixo:



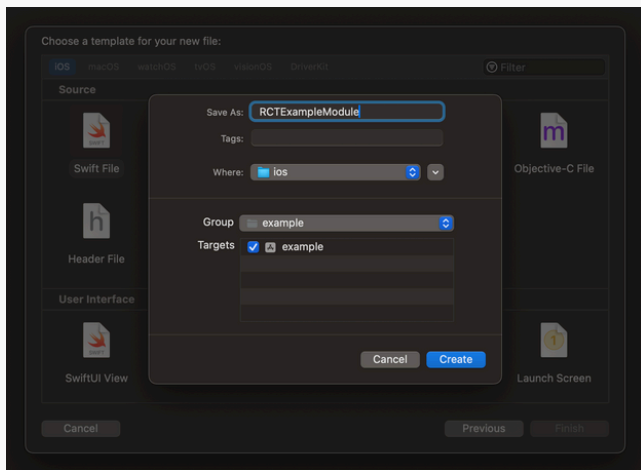
3. Na janela que se abre, selecione o tipo de arquivo "Swift File" e clique em "Next"



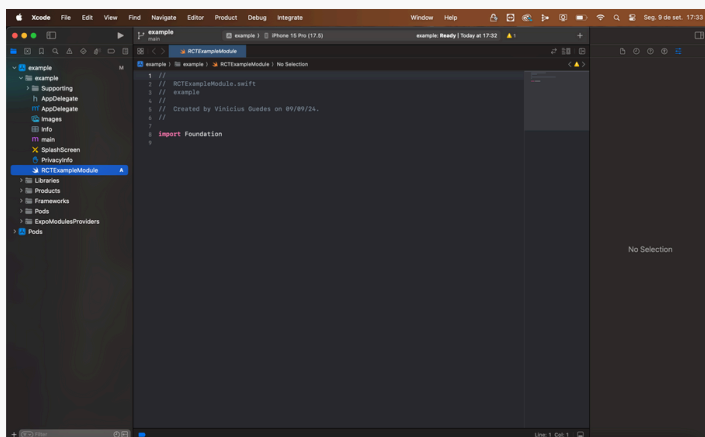
4. Em seguida, será visualizado as opções do arquivo,

- Save As: nomeamos para RCTExampleModule. Definimos esse nome pelo seguinte padrão:
RCT(React)
Example(Exemplo de nome). Altere para a função determinada do módulo
Module(Módulo nativo)
- Where: Local que o arquivo será salvo
- Group: Grupo em que pertence
- Targets: Projeto destinado

Para este passo, nomeie o arquivo de acordo a necessidade. Após, clique em "Create".



Após criado, você terá um arquivo como o mostrado abaixo.



2.1 ESTRUTURA BÁSICA

RCTExampleModule.swift

```
example ) example ) RCTExampleModule ) No Selection
1 //
2 // RCTExampleModule.swift
3 // example
4 //
5 // Created by Vinicius Guedes on 09/09/24.
6 //
7
8 import Foundation
9
```

Se você analisar a estrutura inicial, verá que ela inicia com alguns comentários. Esse é um padrão de criação de arquivos do XCode, no qual sempre inicia seus arquivos com as seguintes informações:

- Nome do arquivo
- Nome do projeto
- Por quem foi criado
- Data de criação

import Foundation

Foundation é um framework fundamental da Apple que fornece muitas funcionalidades básicas para o desenvolvimento em Swift e Objective-C. Ele inclui classes e APIs para trabalhar com strings, números, datas, e também funcionalidades para comunicação em rede, manipulação de arquivos, e muito mais.

Aqui, o import Foundation é necessário para que você possa utilizar recursos e tipos fornecidos pelo framework Foundation na sua classe.

2.2 CRIANDO CLASSE

Um passo essencial para trabalhar com uma linguagem de programação é saber trabalhar com funções. E antes de criarmos nossa função precisamos criar nossa classe.

Veja abaixo como você pode criar sua classe.

RCTExampleModule.swift

```
1 //
2 // RCTExampleModule.swift
3 // example
4 //
5 // Created by Vinicius Guedes on 09/09/24.
6 //
7
8 import Foundation
9
10 class RCTExampleModule {
11
12 }
13
```

class RCTExampleModule:

class é uma palavra-chave em Swift usada para definir uma classe. Uma classe é uma estrutura de dados que pode conter propriedades e métodos que definem seu comportamento e estado.

RCTExampleModule é o nome da classe que está sendo definida. Na prática, esse nome pode ser qualquer nome válido em Swift, mas é comum usar nomes descritivos e relacionados ao propósito da classe.

2.3 CRIANDO FUNÇÃO

Agora que nossa classe foi criada, podemos trabalhar com a criação de nossa função. Vamos criar uma função simples, apenas printar no terminal uma mensagem.

Veja abaixo como você pode criar sua função.

RCTExampleModule.swift

```
1 //  
2 // RCTExampleModule.swift  
3 // example  
4 //  
5 // Created by Vinicius Guedes on 09/09/24.  
6 //  
7  
8 import Foundation  
9  
10 class RCTExampleModule {  
11     func printMessage() {  
12         print("Native message with React Native")  
13     }  
14 }  
15 |
```

func printMessage()

Essa linha define um método chamado printMessage. Este é um método da classe RCTExampleModule que pode ser chamado internamente no código Swift.

print("message text")

print escreve no log do Xcode enquanto o aplicativo está sendo executado. É uma maneira útil de debugar ou verificar se um método nativo foi chamado corretamente.

2.4 EXPONDO FUNÇÕES PARA O REACT NATIVE

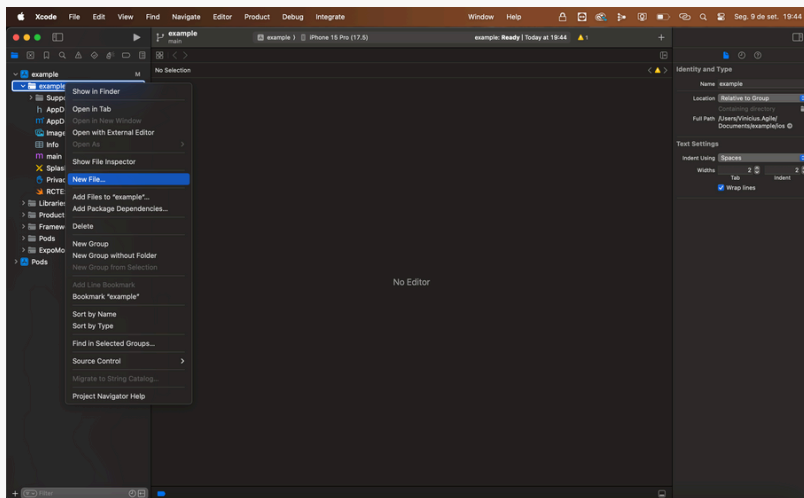
Quando você cria uma classe em Swift para ser usada como um módulo nativo no React Native, é necessário expor essa classe e suas funções ao React Native usando o Objective-C, pois o React Native depende do runtime do Objective-C para interagir com o código nativo.

No React Native, mesmo se você estiver utilizando Swift, a exposição dos métodos e das classes ao JavaScript deve ser feita através de uma interface que o React Native entenda, que é fornecida pelo Objective-C. Para isso, você precisa seguir os seguintes passos:

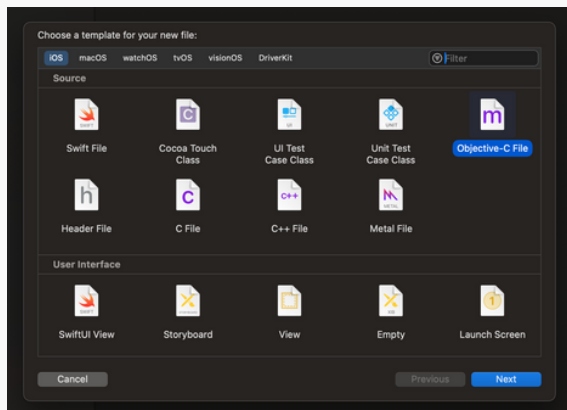
Criar o arquivo de ponte entre Swift e Objective-C:

- O React Native usa Objective-C para se comunicar com o JavaScript, então você precisa de um arquivo de ponte que ajude a conectar o Swift e o runtime do Objective-C.
- Esse arquivo será automaticamente solicitado quando você adicionar código Swift ao projeto.

1. Novamente, no Xcode, entre na pasta SEU_PROJETO/SEU_PROJETO e clique com o botão direito do mouse, após isso, selecione a opção "New File...". Confira na imagem abaixo:



2. Ao abrir as opções, escolha "Objective-C File". Em seguida clique em "Next".

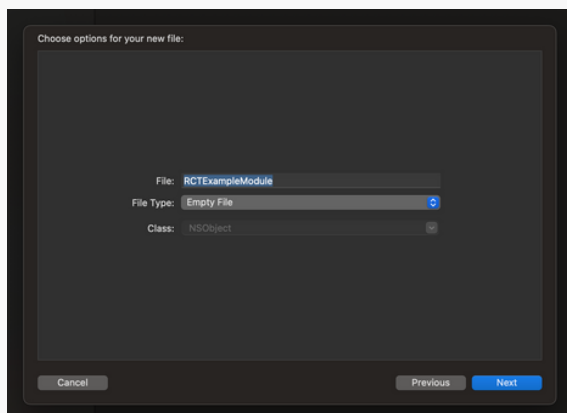


3. Em seguida, nas opções do arquivo:

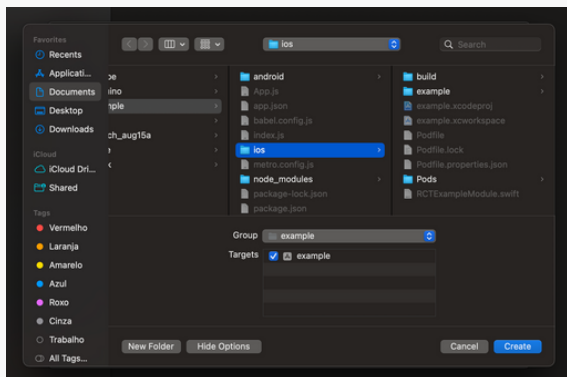
File: Geralmente adotamos RCTExampleModule (Mesmo padrão que você aprendeu anteriormente)

File: Escolha Empty File

Class: NSObject



4. Em seguida, escolha o local em que deseja salvar o novo arquivo, geralmente salvamos junto ao arquivo correspondente do Swift (que criamos na etapa anterior). Em Group selecione o seu projeto, e em Targets escolha o projeto destino, geralmente existe apenas um, que é o projeto que você está trabalhando.



Ao final, você verá algo parecido como a imagem abaixo em seu novo arquivo:

```
1 //  
2 // RCTExampleModule.m  
3 // example  
4 //  
5 // Created by Vinicius Guedes on 09/09/24.  
6 //  
7  
8 #import <Foundation/Foundation.h>  
9
```

Agora que já temos o arquivo necessário para expor nossa classe, podemos trabalhar com ela. O primeiro passo é remover a importação do Foundation (linha 8 da imagem anterior), pois não precisaremos dela neste arquivo.

Agora iremos expor nosso módulo Swift.

5. Para expor um módulo Swift através do Objective-C para o React Native, você precisará dessa estrutura mínima, ela que permitirá que o JavaScript interaja com funcionalidades nativas do iOS.

Em seu `RCTExampleModule.m` escreva o seguinte código:

`RCTExampleModule.m`

```
1 //
2 // RCTExampleModule.m
3 // example
4 //
5 // Created by Vinicius Guedes on 09/09/24.
6 //
7
8 #import "React/RCTBridgeModule.h"
9
10 @interface RCT_EXTERN_MODULE(RCTExampleModule, NSObject)
11
12 @end
13 |
```

Veja detalhadamente sobre cada linha de código:

`#import "React/RCTBridgeModule.h"`

Esse comando importa o cabeçalho `RCTBridgeModule.h`, que é parte do React Native e contém a definição da interface necessária para criar módulos nativos que possam ser utilizados pelo JavaScript no React Native. O `RCTBridgeModule` é um protocolo que qualquer módulo nativo precisa implementar para ser registrado e reconhecido pelo React Native.

Resumindo, essa linha permite que a classe Swift que será exposta (via Objective-C) interaja com o sistema de bridge do React Native.

`@interface RCT_EXTERN_MODULE(RCTExampleModules, NSObject)`

@interface: Define uma interface no Objective-C. No caso, estamos declarando a interface de um módulo nativo que será exposto ao React Native.

RCT_EXTERN_MODULE: Essa macro (função especial) do React Native é usada para expor um módulo Swift ao React Native através do Objective-C. Ela cria automaticamente a implementação necessária para que o código Swift seja acessível pelo runtime do Objective-C.

RCTExampleModules: Esse é o nome do módulo que estamos expondo ao React Native. Ele corresponde à classe Swift que você criou. Esse nome será usado no código JavaScript para acessar as funcionalidades do módulo.

NSObject: Especifica que a classe `RCTExampleModules` herda de `NSObject`, o que é necessário para compatibilidade com o runtime do Objective-C, além de garantir que o módulo possa ser registrado corretamente no sistema do React Native.

`@end`

Esta linha encerra a declaração da interface no Objective-C. Ela indica que todas as exposições feitas pela interface estão completas.

6. Com nossa classe `RCTExampleModule` sendo exposta, agora precisamos expor as funções que estão dentro da nossa classe.

Para isso, escreva o seguinte código em destaque (linha 12):

`RCTExampleModule.m`

```
example c example RCTExampleModule No Selection
1 //
2 // RCTExampleModule.m
3 // example
4 //
5 // Created by Vinicius Guedes on 09/09/24.
6 //
7
8 #import "React/RCTBridgeModule.h"
9
10 @interface RCT_EXTERN_MODULE(RCTExampleModule, NSObject)
11     RCT_EXTERN_METHOD(printMessage); // Adicione esta linha de código
12
13 @end
14
15
```

Vamos entender o que fizemos:

`RCT_EXTERN_METHOD`

É uma macro fornecida pelo React Native que diz ao sistema de bridge para expor um método de uma classe Swift (No nosso caso, a classe `RCTExampleModule`) para o runtime do Objective-C, e consequentemente, para o JavaScript. Ele faz isso sem exigir que você precise escrever código Objective-C completo para a implementação, o que torna o processo mais simples.

`printMessage`

Este é o nome do método que está sendo exposto. No Swift, a nossa função que mostra o log no terminal.

7. Agora que estamos expondo para o React Native nossa classe e função, precisamos antes expor para o Objective-C a nossa classe e função Swift. Então agora vamos voltar ao nosso arquivo RCTExampleModule.swift

Em seu arquivo RCTExampleModule.swift adicione os códigos em destaque (Linha 10 e linha 12):

RCTExampleModule.swift

```
1 //
2 // RCTExampleModule.swift
3 // example
4 //
5 // Created by Vinicius Guedes on 09/09/24.
6 //
7
8 import Foundation
9
10 @objc(RCTExampleModule) // Adicione esta linha
11 class RCTExampleModule: NSObject {
12     @objc // Adicione esta linha
13     func printMessage() {
14         print("Native message with React Native")
15     }
16 }
17
```

Vamos entender o que fizemos:

@objc(RCTExampleModule)

@objc: Essa anotação transforma a classe Swift em uma classe compatível com o runtime do Objective-C. Para que o React Native possa interagir com classes e métodos em Swift, eles precisam ser expostos ao runtime do Objective-C, que é o sistema que o React Native utiliza para comunicar o JavaScript com o código nativo. O @objc essencialmente "marca" a classe como disponível para o mundo Objective-C.

(RCTExampleModule): O parâmetro entre parênteses define o nome com o qual essa classe será registrada no runtime do Objective-C e, por extensão, como será exposta ao React Native. Aqui, você está especificando que o nome da classe será RCTExampleModule. Este será o nome que você usará para acessá-la do JavaScript no React Native. Se não passasse um nome, a classe Swift manteria seu nome original.

@objc

Essa anotação é aplicada ao método printMessage, tornando-o disponível para o runtime do Objective-C, assim como a anotação anterior fez para a classe. O React Native pode acessar e invocar apenas métodos que tenham sido expostos ao Objective-C, e a anotação @objc faz essa ponte.

Sem essa anotação, o método printMessage seria acessível apenas internamente no Swift e não poderia ser chamado a partir do React Native. Ao usar @objc, você garante que o método pode ser chamado de JavaScript no React Native.

ATÉ AQUI VOCÊ APRENDEU

Vamos recapitular o que fizemos até aqui.

01

Classe e função Swift

RCTExampleModule.swift

Isso estabelece o código nativo Swift que você deseja expor ao React Native. Inicialmente, você cria a classe e os métodos nativos

02

Expor Classe

RCTExampleModule.h

Você cria o arquivo .h (Objective-C) e usa a macro `RCT_EXTERN_MODULE` para expor a classe Swift ao sistema de bridge do React Native, permitindo que o React Native reconheça o módulo nativo.

03

Expor Função

RCTExampleModule.h

No mesmo arquivo .h, você utiliza a macro `RCT_EXTERN_METHOD` para expor os métodos da classe Swift, possibilitando que eles sejam chamados diretamente do JavaScript no React Native.

04

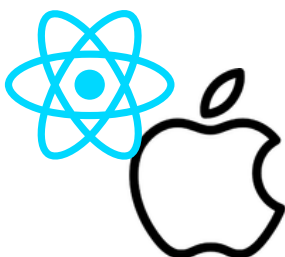
@objc

RCTExampleModule.swift

Depois de expor o módulo, você volta ao arquivo Swift e adiciona a anotação `@objc` tanto na classe quanto nos métodos. Isso garante que o runtime do Objective-C possa acessar o código Swift.

Conexão React Native

CAPÍTULO 3



Bridge iOS

3. ACESSANDO MÓDULO NATIVO

Finalmente, com nosso código nativo desenvolvido podemos acessar através do React Native. Para isso, iremos precisar usar o `NativeModules` do React Native para fazer essa bridge com nosso código nativo.

Em nosso projeto React Native, acesse o arquivo `App.js` na raiz do projeto, e adicione o código abaixo:

App.js

```
App.js > ...
You, 7 minutes ago | 1 author (You)
1 import React from 'react';
2 import { StyleSheet, Text, View, NativeModules, Button } from 'react-native';
3
4 export default function App() {
5   const { ExampleModule } = NativeModules;
6   return (
7     <View style={styles.container}>
8       <Text>Native module app!</Text>
9       <Button title="Click me for printMessage" onPress={() => ExampleModule.printMessage()} />
10     </View>
11   );
12 }
13
14 const styles = StyleSheet.create({
15   container: {
16     flex: 1,
17     backgroundColor: '#fff',
18     alignItems: 'center',
19     justifyContent: 'center',
20   },
21 });
```

Vamos entender o que fizemos:

`const { ExampleModule } = NativeModules;`

NativeModules: É um objeto fornecido pelo React Native que contém todos os módulos nativos expostos ao JavaScript. Ele permite que o código JavaScript acesse e interaja com funcionalidades nativas do sistema operacional, que foram registradas via bridge.

{ ExampleModule }: Utiliza destructuring assignment para extrair o módulo nativo chamado `ExampleModule` de dentro de `NativeModules`. Nesse caso, `ExampleModule` é o nome que foi registrado anteriormente (via Swift e Objective-C no iOS ou via Kotlin/Java no Android).

Quando você usa a anotação `@objc(RCTExampleModule)` na declaração da classe Swift, você está expondo o nome `RCTExampleModule` para o runtime do Objective-C. Porém, quando o módulo é registrado no JavaScript, o React Native remove automaticamente o prefixo `RCT`.

ExampleModule.printMessage()

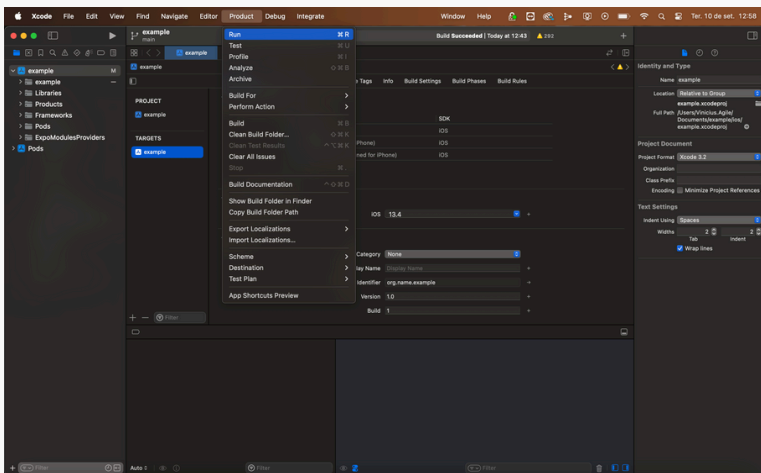
printMessage: Função swift criada e exposta pelo Objective-C, conforme criamos, ela será responsável por imprimir no terminal do XCode uma mensagem.

3.1 TESTANDO MODIFICAÇÕES

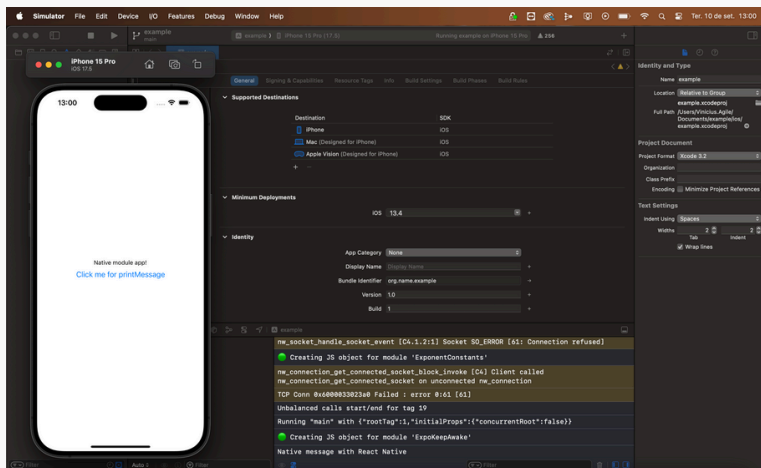
Finalmente chegou o momento mais esperado do nosso projeto. Vamos testar o que fizemos

Novamente, entre na pasta `SEU_PROJETO/ios` e abra o arquivo `SEU_PROJETO.xcworkspace`. Feito isso, rode via xcode o seu app através do caminho *Product > run*.

Lembre-se antes de instalar os pods, e rodar o comando `yarn start` ou `npm run start`.

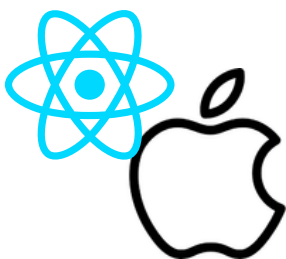


Ao rodar o app e pressionar o botão "Click me for printMessage", você verá no terminal do Xcode nossa mensagem "Native message with React Native"



Envio de Parâmetros

CAPÍTULO 4



Bridge iOS

4 ENVIO DE PARÂMETROS

O envio de parâmetros é algo essencial em uma bridge, aliás, é de suma importância sabermos enviar parâmetros pois será dessa forma que poderemos manipular e transportar dados Javascript para a linguagem nativa Swift. Assim, poderemos via Javascript controlar nativamente nosso aplicativo e acessar/controlar recursos nativos complexos.

1. Vamos então agora realizar pequenas modificações no que fizemos até o momento. Vamos começar pelo RCTExampleModule.swift

RCTExampleModule.swift

```
1 //  
2 // RCTExampleModule.swift  
3 // example  
4 //  
5 // Created by Vinicius Guedes on 09/09/24.  
6 //  
7  
8 import Foundation  
9  
10 @objc(RCTExampleModule)  
11 class RCTExampleModule: NSObject {  
12     @objc  
13     func printMessage(_ title: String) { // Altere esta linha  
14         print("Native message with React Native ") // Altere esta linha  
15     }  
16 }  
17
```

Vamos entender o que fizemos:

func printMessage(_ title: String)

_: Usar o sublinhado (_) omite o nome do parâmetro ao chamar a função. Isso significa que, quando você chamar printMessage, não será necessário especificar o nome do parâmetro. No lugar de printMessage(title: "Texto"), você simplesmente usa printMessage("Texto").

title: É o nome do parâmetro que a função recebe. Nesse caso, ele representa o valor que será passado para a função, que pode ser uma string.

String: Tipo de dado do parâmetro title

print("Native message with React Native \"(title)\")

\(title): Estamos concatenando o texto que vem via parâmetro ao texto nativo da nossa função.

2. Agora precisamos também expor dentro do `RCTExampleModule.m` que nossa função `printMessage` recebe um parâmetro. Em `RCTExampleModule.m` Adicione o seguinte código em destaque:

`RCTExampleModule.m`

```
example ) example ) RCTExampleModule ) No Selection
1 //
2 // RCTExampleModule.m
3 // example
4 //
5 // Created by Vinicius Guedes on 09/09/24.
6 //
7
8 #import "React/RCTBridgeModule.h"
9
10 @interface RCT_EXTERN_MODULE(RCTExampleModule, NSObject)
11
12     RCT_EXTERN_METHOD(printMessage:(NSString *) title); // Altere esta linha de código
13
14 @end
15 |
```

Vamos entender o que fizemos:

`(NSString *) title`

Define o tipo do parâmetro que o método aceita. Aqui, o parâmetro `title` é do tipo `NSString`, que é a representação da string no Objective-C (e também compatível com Swift).

3. Podemos agora enviar via parâmetro um valor do tipo string para nossa função nativa. Em `App.js`, adicione o seguinte código em destaque:

`App.js`

```
App.js > ...
You, 24 seconds ago (1 author) (You)
1 import React from 'react';
2 import { StyleSheet, Text, View, NativeModules, Button } from 'react-native';
3
4 export default function App() {
5   const { ExampleModule } = NativeModules;
6   return (
7     <View style={styles.container}>
8       <Text>Native module app!</Text>
9       <Button
10         title="Click me for printMessage"
11         onPress={() => ExampleModule.printMessage("My title")} // Altere esta linha de código
12       />
13     </View>
14   );
15 }
16
17 const styles = StyleSheet.create({
18   container: {
19     flex: 1,
20     backgroundColor: 'pink',
21     alignItems: 'center',
22     justifyContent: 'center',
23   },
24 });
25
```

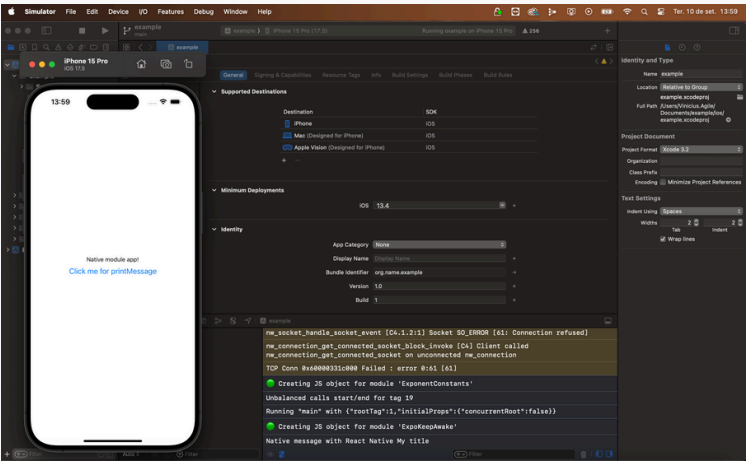
Vamos entender o que fizemos:

`ExampleModule.printMessage("My title")`

Adicionamos o parâmetro que nossa função nativa swift está esperando.

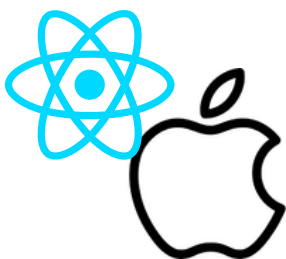
4. Agora vamos novamente via XCode rodar o nosso app conforme você aprendeu anteriormente.

Agora, ao clicar no botão, você verá a saída "Native message with React Native My title" que é a concatenação de "Native message with React Native" que é o texto que estamos imprimindo nativamente com "My title" que o texto que vem como parâmetro da função chamada no botão React Native.



Retorno de Função

CAPÍTULO 5



Bridge iOS

5 RETORNO DE FUNÇÃO

O ideal de uma conexão é que seja bidirecional, ou seja, ela envia informações e também recebe informações. Isso nos ajuda a criar aplicações e funcionalidades complexas e robustas. Uma bridge também deve conter uma comunicação bidirecional. E é sobre isso que iremos aprender nesse capítulo

1. Vamos então agora realizar novas modificações no que fizemos até o momento. Vamos começar pelo `RCTExampleModule.swift`. Adicione os códigos em destaque:

`RCTExampleModule.swift`

```
1 //
2 // RCTExampleModule.swift
3 // example
4 //
5 // Created by Vinicius Guedes on 09/09/24.
6 //
7
8 import Foundation
9 import React // Adicione esta linha
10
11 @objc(RCTExampleModule)
12 class RCTExampleModule: RCTEventEmitter {
13     @objc
14     func printMessage(_ title: String) {
15         print("Native message with React Native \(title)")
16     }
17
18     // Adicione esta nova função
19     @objc
20     func returnMessage(_ title: String, resolver resolve: @escaping RCTPromiseResolveBlock, rejecter reject:
21         @escaping RCTPromiseRejectBlock) {
22         resolve("Message processed: \(title)")
23     }
24 }
```

Vamos entender o que fizemos:

`import React`

Importa as APIs do React Native para interagir com o sistema de bridge, como o suporte a promessas com `RCTPromiseResolveBlock` e `RCTPromiseRejectBlock`

`func returnMessage(_ title: String, resolver resolve: @escaping RCTPromiseResolveBlock, rejecter reject: @escaping RCTPromiseRejectBlock)`

_ title: String: Um parâmetro do tipo String que é passado a partir do código JavaScript. Ele contém a mensagem que será processada.

resolver resolve: RCTPromiseResolveBlock: Um bloco de resolução que será chamado quando a operação for bem-sucedida, retornando dados para o código JavaScript como uma Promise resolvida.

rejecter reject: RCTPromiseRejectBlock: Um bloco de rejeição que será chamado em caso de erro, retornando uma Promise rejeitada ao código JavaScript.

`resolve("Message processed: \$(title)")`

Este comando resolve a Promise com a mensagem processada. A string "Message processed: \\$(title)" é enviada de volta ao React Native como resposta.

O padrão de resolver ou rejeitar promessas é utilizado para facilitar a interação assíncrona entre o código nativo (Swift) e o código React Native (JavaScript). No lado do JavaScript, você pode usar o método `returnMessage` com `async/await` ou `.then()/.catch()`.

2. Precisamos agora expor essa nova função Swift no Objective-C para que seja entendido no React Native. Adicione o código em destaque:

RCTExampleModule.m

```
example 1 example 1 RCTExampleModule No Selection
1 //
2 // RCTExampleModule.m
3 // example
4 //
5 // Created by Vinicius Guedes on 09/09/24.
6 //
7
8 #import "React/RCTBridgeModule.h"
9
10 @interface RCT_EXTERN_MODULE(RCTExampleModule, NSObject)
11
12 RCT_EXTERN_METHOD(printMessage:(NSString *) title);
13 RCT_EXTERN_METHOD(returnMessage: (CGFloat *)title resolver:(RCTPromiseResolveBlock)resolve
14                   rejecter:(RCTPromiseRejectBlock)reject) // Adicione esta linha de código
15
16 @end
```

Você já conhece sobre o `RCT_EXTERN_METHOD`, `returnMessage` e o `(NSString *) title`. Vamos entender então o outro trecho do código

`resolver:(RCTPromiseResolveBlock)resolve`

Define um segundo parâmetro chamado `resolver`, do tipo `RCTPromiseResolveBlock`. Esse parâmetro é usado para resolver a Promise no JavaScript, ou seja, quando a operação for bem-sucedida, ele retornará o resultado.

`rejecter:(RCTPromiseRejectBlock)reject`

Define um terceiro parâmetro chamado `rejecter`, do tipo `RCTPromiseRejectBlock`. Esse parâmetro é usado para rejeitar a Promise no JavaScript, caso ocorra um erro durante a execução da função.

3. Agora em nosso App.js podemos realizar a alteração que precisamos para capturar o retorno de uma chamada. Adicione os códigos em destaque.

App.js

```
App.js
1  import React from 'react';
2  import { StyleSheet, Text, View, NativeModules, Button } from 'react-native';
3
4  export default function App() {
5    const { ExampleModule } = NativeModules;
6    return (
7      <View style={styles.container}>
8        <Text>Native module app!</Text>
9        <Button
10          title="Click me for printMessage"
11          onPress={() => ExampleModule.printMessage("My title")}
12        />
13      </View>
14    );
15    // Adicione este trecho de código a/b
16    <Button title="Click me for returnMessage" onPress={() =>
17      ExampleModule.returnMessage("My title").then(result) => {
18        console.log('Mensagem recebida do Swift:', result);
19      }
20    } />
21  );
22  </View>
23  );
24  }
25
26  const styles = StyleSheet.create({
27    container: {
28      flex: 1,
29      backgroundColor: '#fff',
30      alignItems: 'center',
31      justifyContent: 'center',
32    },
33  });
```

Vamos entender o que fizemos:

ExampleModule.returnMessage('My title')

ExampleModule: Refere-se ao módulo nativo que foi exposto pelo sistema de bridge do React Native. Esse módulo foi importado usando NativeModules e corresponde à classe Swift (ou Objective-C) RCTExampleModule.

returnMessage('My title'): Está chamando o método returnMessage do módulo nativo, passando a string 'My title' como argumento. No código nativo, essa string é recebida como um parâmetro (title) e processada. O método returnMessage retorna uma Promise para o código JavaScript.

.then((result) => { ... })

.then(): É parte da estrutura de Promises em JavaScript. Ele define o que acontecerá quando a Promise for resolvida com sucesso. Isso significa que o código Swift executou o método sem erros e retornou um resultado.

(result) => { console.log('Mensagem recebida do Swift:', result); }: Essa função é executada quando a Promise é resolvida. O parâmetro result contém o valor retornado pelo método nativo Swift. Nesse caso, a função exibe no console o texto "Mensagem recebida do Swift:" seguido do resultado retornado pelo Swift.

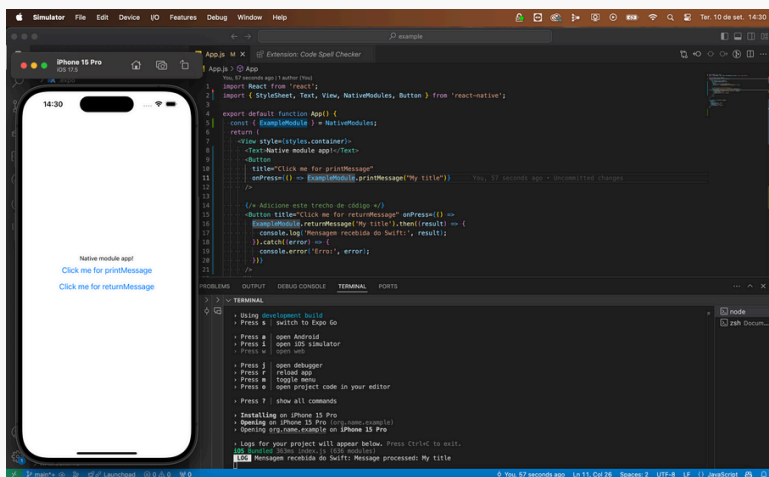
.catch((error) => { ... })

.catch(): Também faz parte da estrutura de Promises. Ele define o que acontecerá se a Promise for rejeitada, ou seja, se ocorrer algum erro ao executar o método nativo.

(error) => { console.error('Erro:', error); }: Essa função é chamada quando a Promise é rejeitada. O parâmetro error contém detalhes do erro que ocorreu. Nesse caso, ele exibe no console a mensagem "Erro:" seguida do conteúdo do erro.

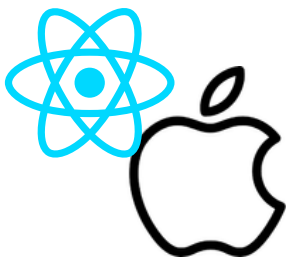
4. Como não precisamos mais visualizar logs no XCode e agora precisamos visualizar no terminal do metro bundle do nosso projeto, Vamos apenas rodar o comando `yarn ios` ou `npm run ios` no terminal do nosso VSCode.

Após rodar o seu app e pressionar o botão "Click me for returnMessage" você verá no terminal do VSCode a mensagem "Mensagem recebida do Swift: Message processed: My title" Esta mensagem é a concatenação de "Mensagem recebida do Swift:" (log da linha 17) com "Message processed: " (Texto nativo da função swift) e "My title" (Parâmetro enviado para nossa função nativa)



Event Listener

CAPÍTULO 6



Bridge iOS

6 RETORNO EVENT LISTENER

Eventos listeners são também uma ótima forma de trabalharmos com bridge, isso porque podemos manipular dados assíncronos de forma independente em nosso código nativo enquanto ouvimos e aguardamos suas alterações do lado React Native. Vamos aprender como podemos fazer isso.

1. Vamos começar pelo RCTExampleModule.swift.

Adicione o seguinte código em destaque:

RCTExampleModule.swift

```
1 //  
2 // RCTExampleModule.swift  
3 // example  
4 //  
5 // Created by Vinicius Guedes on 09/09/24.  
6 //  
7  
8 import Foundation  
9 import React  
10  
11 @objc(RCTExampleModule)  
12 class RCTExampleModule: RCTEventEmitter { // Altere esta linha de código  
13 @objc  
14 func printMessage(_ title: String) {  
15     print("Native message with React Native \(title)")  
16 }  
17  
18 @objc  
19 func returnMessage(title: String, resolver resolve: @escaping RCTPromiseResolveBlock, rejecter reject: @escaping  
20     RCTPromiseRejectBlock) {  
21     resolve("Message processed: \(title)")  
22 }  
23  
24 // Adicione esta função  
25 @objc  
26 func eventMessage(_ value: @escaping RCTPromiseResolveBlock) {  
27     sendEvent(withName: "onMessagePrinted", body: ["value": value])  
28 }  
29  
30 // Adicione esta função  
31 override func supportedEvents() -> [String]! {  
32     return ["onMessagePrinted"]  
33 }  
34 }
```

Vamos entender o que fizemos:

class RCTExampleModule: RCTEventEmitter

class RCTExampleModule: RCTEventEmitter: Aqui alteramos de NSObject para RCTEventEmitter, isso para definir o RCTExampleModule como uma subclasse de RCTEventEmitter, o que permite que ela emita eventos nativos que possam ser escutados no React Native.

RCTEventEmitter é uma classe que oferece suporte para emitir eventos do código nativo para o JavaScript. Herdar dela é necessário quando queremos usar métodos como sendEvent para enviar informações para o React Native.

eventMessage

@objc func eventMessage(_ value: CGFloat): Define uma função chamada eventMessage, que recebe um valor do tipo CGFloat (um número de ponto flutuante) como parâmetro.

sendEvent(withName: "onMessagePrinted", body: ["value": value]): Dentro dessa função, o método sendEvent é usado para emitir um evento chamado "onMessagePrinted". O evento inclui um corpo (body), que é um dicionário com a chave "value" e o valor passado pela função.

Esse evento será enviado para o JavaScript, e o React Native poderá reagir a ele através de event listeners.

supportedEvents

override func supportedEvents() -> [String]!: Esta função sobrescreve o método supportedEvents da classe RCTEventEmitter. Ela retorna um array de strings que contém os nomes dos eventos que este módulo pode emitir.

return ["onMessagePrinted"]: Aqui, a função define que o único evento que este módulo suporta é "onMessagePrinted". Este é o nome que o código JavaScript usará para escutar os eventos emitidos a partir do código nativo. Você pode alterar o onMessagePrinted para um nome de evento que desejar.

Quando a função eventMessage é chamada, o código Swift dispara o evento "onMessagePrinted" e envia o valor recebido pelo parâmetro.

No lado do React Native, será necessário escutar o evento "onMessagePrinted" para capturar e manipular o valor enviado do código nativo.

2. Agora vamos expor nossa nova função em RCTExampleModule.m. Adicione o seguinte trecho de código em destaque:

RCTExampleModule.m

```
1 //
2 // RCTExampleModule.m
3 // example
4 //
5 // Created by Vinicius Guedes on 09/09/24.
6 //
7
8 #import "React/RCTBridgeModule.h"
9
10 @interface RCT_EXTERN_MODULE(RCTExampleModule, NSObject)
11
12   RCT_EXTERN_METHOD(printMessage:(NSString *) title);
13   RCT_EXTERN_METHOD(returnMessage:(NSString *)title resolver:(RCTPromiseResolveBlock)resolve
14                       rejecter:(RCTPromiseRejectBlock)reject)
15   RCT_EXTERN_METHOD(eventMessage: (CGFloat *) value); // Adicione esta linha de código
16
17 @end
18
```

Vamos entender o que fizemos:

RCT_EXTERN_METHOD(eventMessage: (CGFloat *) value);

Aqui a única diferença do que vimos até agora é a adição do CGFloat, que é uma declaração que o parâmetro esperado é do tipo numérico.

3. Por último, vamos alterar o nosso App.js. Adicione os códigos em destaque:

App.js

```
1 import { StyleSheet, Text, View, NativeModules, Button, NativeEventEmitter } from 'react-native';
2
3
4 export default function App() {
5   const { ExampleModule } = NativeModules;
6   const eventEmitter = new NativeEventEmitter(ExampleModule); // Adicione este trecho de código
7
8   // Adicione esta função
9   const subscription = eventEmitter.addListener('onMessagePrinted', (event) => {
10     console.log("Evento recebido:", event.value);
11   });
12
13   return (
14     <View style={styles.container}>
15       <Text>Native module app!</Text>
16       <Button
17         title="Click me for printMessage"
18         onPress={() => ExampleModule.printMessage("My title")}
19       />
20
21       <Button title="Click me for returnMessage" onPress={() =>
22         ExampleModule.returnMessage("My title").then((result) => {
23           console.log "Mensagem recebida do Swift:", result;
24         }).catch((error) => {
25           console.error('Error:', error);
26         })
27       />
28
29       </* Adicione este trecho de código */>
30       <Button
31         title="Click me for eventMessage"
32         onPress={() => ExampleModule.eventMessage(3)}
33       />
34     </View>
35   );
36 }
37
38 const styles = StyleSheet.create({
39   container: {
40     flex: 1,
41     backgroundColor: 'fff',
42     alignItems: 'center',
43     justifyContent: 'center',
44   },
45 });
46 }
```

Vamos entender o que fizemos:

const eventEmitter = new NativeEventEmitter(ExampleModule);

Cria uma instância de NativeEventEmitter associada ao ExampleModule. Isso permite que o JavaScript escute eventos emitidos por ExampleModule.

const subscription

eventEmitter.addListener('onMessagePrinted', ...): Adiciona um listener que escuta eventos com o nome "onMessagePrinted" emitidos pelo ExampleModule.

(event) => { console.log("Evento recebido:", event.value) }: Função callback que é chamada sempre que o evento "onMessagePrinted" é emitido. Ela simplesmente imprime o valor recebido no console.

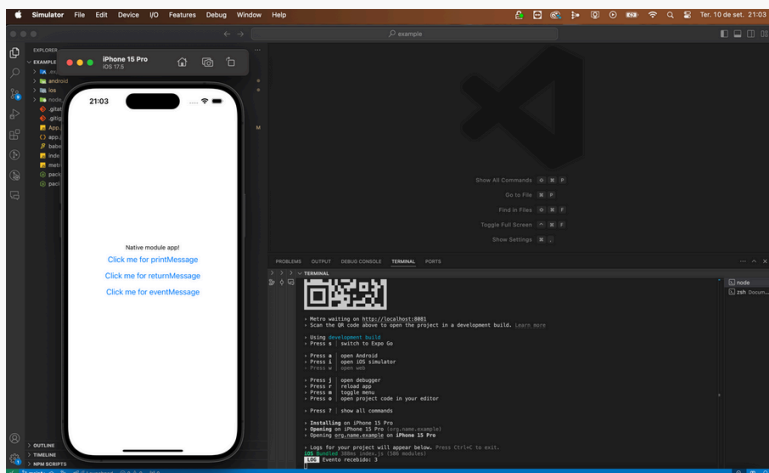
ExampleModule.eventMessage(3)

Dispara nossa função nativa eventMessage enviando como parâmetro o valor numérico 3.

Importante: Não esqueça de remover o listener quando não for mais necessário ouvir modificações. Isso é importante para evitar problemas de performance do seu aplicativo.

Para remover o listener basta usar o comando **subscription.remove()**.

4. Após novamente rodar o seu aplicativo e pressionar o botão "lick me for eventMessage" você verá no terminal do VSCode a mensagem "Evento recebido: 3". Esse é o retorno do listener que estamos escutando em na constant subscription



Resumo

CAPÍTULO 7

7 RESUMO

Neste ebook, fornecemos uma base sólida para conectar React Native com código nativo em iOS. Você aprendeu a criar funções nativas, enviar e receber dados, e gerenciar eventos, o que é crucial para construir aplicações móveis robustas e interativas. Com essas habilidades, você está apto a integrar e utilizar o código nativo de forma eficiente, ampliando as possibilidades do seu desenvolvimento com React Native.

Agradecemos por acompanhar este guia e esperamos que as técnicas e conceitos apresentados aqui sejam valiosos para seus projetos futuros. Continue explorando e inovando com React Native e iOS!